

Take-Home Assignment: SQL Query Application

Objective:

Develop a system that allows users to query a SQL database using natural language queries. The system will process a dataset provided as a CSV file (`data.csv`), load it into a SQL database, and use a pre-trained text-to-SQL model to translate natural language queries into SQL. The application will execute the SQL queries on the database and return a human-readable answer via a user interface.

Tasks:

Database Service: load the provided CSV dataset into a SQL database and expose the database via an API endpoint for querying.

Text-to-SQL Model Service: utilize a pre-trained text-to-SQL model to translate natural language queries into SQL. You may use an open-source or pre-trained model such as SQLCoder-2-7b, T5-base, any Hugging Face models, an SLM through Ollama, or any other option you see fit. **This service should HOST the aforementioned model and output inference through it.** Expose an API endpoint to receive a natural language query and any other endpoint you believe necessary.

User Interface Service: create a user interface (web or API-based) where users can input a natural language query (e.g., "What is the most bought product on Fridays?"). Display the sql result after processing.

Bonus task [OPTIONAL] - LLM/SLM Model service: host a second model, an LLM/SLM that will translate the query results into human readable language making the system transparent to the end user. This means, the user asks in natural language "What is the most bought product on Fridays?" and receives after some seconds "Alfajor 70 cacao x un is the most bought product on Fridays" written in natural language.

Requirements:

- Code should be in **Python** (the user interface doesn't have to if you choose to create a web frontend).
- You may implement as many services as needed. Either a service per task, one service (monolith) for all the tasks or how many you deem fit.
Simplicity is highly desirable and valuable. (KISS principle).
- Each service, job, or database should run within its own Docker container.
- Use Docker Compose to orchestrate and run all services locally.
- Ensure proper communication between services, using networking or shared volumes as needed.
- **Provide clear documentation and setup instructions** (README.md is a good place for this)

Deliverables:

- Source code for all services. Preferably a github link (other repos are also fine).
- A **docker-compose.yml** file in the repo to run all services locally.
- Instructions on how to build and run the application.
- You should be able to explain how you would scale out the architecture if there were bigger and more tables; and also high traffic to the front interface. You can add this as a section to the README.md

Recommendations:

- Architectural decisions and trade-offs matter most: the most important thing I'll evaluate is your architecture and how you manage trade-offs. For example: a smaller model is easier to host and cheaper/faster, but may produce worse SQL. If you choose a small model, explain how you mitigate quality (prompting strategy, constrained decoding, schema injection, examples, validation loops, retries, etc.). If you choose a larger model, it might need less work around quality and more work around making it work in a standard personal computer. Whatever you choose, **the system should actually work end-to-end**.
Please take into account that we will run this on our own machines: the solution should run reliably outside of your local environment. We want to avoid "it runs on my computer" situations — **portability and reproducibility are part of the challenge**. If anything about setup is unclear, feel free to reach out and ask.
- Clean and fast setup is important: the system should start as simply and seamlessly as possible. `docker compose up` is perfectly fine; a few extra steps are also OK if justified. What matters is that **setup is straightforward and the system runs without friction**.
- Product thinking and UX are appreciated: even a simple UI or API should feel intentional: clear inputs, clear outputs, and transparency into what the system is doing (e.g. generated SQL, errors, or results).
- Focus on modularity and clear separation of concerns between services.
- Avoid using an external service API (OpenAI e.g.) for the model part, unless you want to include it as an extra functionality for benchmarking sake. To clarify, if you want to add an option to switch between the model you hosted and OpenAI's API (for example) so comparison becomes possible, you can do it. But **prioritize hosting the model, since that is what I'm interested in**.
- Ensure that services are resilient and handle errors gracefully.
- Write clean, maintainable code with appropriate documentation and comments.
- Start with the simple tasks such as downloading the creating and hosting the db, then move on to more complex ones like dealing with the text-to-sql model.
- We know you will use chatGPT, Claude or any other LLM or AI framework like Cursor or Claude Code as help with this project. **This is not only expected, but encouraged**.
- Feel free to write to me @ alex@nivii.ai or through whatsapp if you have any questions.